

NixOS bootstrap ownership failure — research report

Root cause and repair: nixos-anywhere bootstrap ownership failure

Date: 2026-07-13

Repository: Meillaya/nix-config

Verdict: Root cause confirmed locally across independent source, generated-system, namespace, mutation, and adversarial-security evidence.

Executive finding

The transfer was already correct. The helper and pinned nixos-anywhere set the bootstrap directory and hash to numeric UID/GID 0:0 with modes 0700/0600 before nixos-install. The validator nevertheless compared GNU stat's **owner names** (%U:%G) with the literal string root:root.

On a fresh install, that validator is intentionally ordered before the NixOS users activation fragment. nixos-install creates the target /etc/NIXOS and /etc/mtab, chroots into /mnt, and invokes activation before target /etc/passwd and /etc/group exist. GNU coreutils therefore formats the correct numeric IDs as UNKNOWN:UNKNOWN, making the old name predicate reject a correct inode.

The exact pre-fix behavior was reproduced:

```
numeric=0:0:700 named=UNKNOWN:UNKNOWN:700  
bootstrap password hash validation failed: expected root:root mode  
0700 ...
```

The downstream /run/current-system/bin/switch-to-configuration error is consequential: the validator aborts before activation creates /run/current-system; nixos-enter ignores the preliminary activation status and then tries the missing path.

Causal chain

1. The helper stages a private directory/file and passes numeric --chown var/lib/nixos-bootstrap 0:0.

2. Pinned nixos-anywhere runs extra-files tar extraction, explicit chown, then nixos-install; its fatal shell settings would stop before activation if transfer/chown failed. [Source 1: pinned transfer source](#)
3. Exact repeat-install execution produced numeric 0:0 and modes 700/600; upstream also has an end-to-end numeric chown test. [Source 2: upstream chown test](#)
4. Pinned NixOS creates only the minimal target /etc markers before entering the target root. [Source 3: pinned nixos-install](#)
5. nixos-enter chroots /mnt, invokes the system activation script, ignores its status, then executes the requested switch command. [Source 4: pinned nixos-enter](#)
6. The generated order is bootstrap validator → users → consumer. Before users, target passwd/group resolution is unavailable.
7. Coreutils %u/%g report inode numbers, whereas %U/%G call name lookup and emit UNKNOWN when it fails. [Source 5: GNU coreutils 9.11 stat.c](#)
8. The name predicate exits before /run/current-system is established, causing the observed secondary ENOENT.

Why the prior chown fix did not help

chown 0:0 changes inode metadata. It cannot create target passwd/group entries or change how %U/%G render an otherwise-correct UID/GID before users activation. The prior log also printed only a hard-coded expected tuple, not the tuple actually observed, so it never proved ownership was wrong.

Transfer, wrong filesystem root, mode mutation, destination substitution, missing closure, ACL, ordinary local-filesystem, and later-metadata-mutation hypotheses were counter-searched and rejected for this attempt. Detailed evidence is in the [Wave 1 synthesis](#), [transfer report](#), and [causal skeptic report](#).

Implemented repair

modules/nixos/bootstrap-password.nix now:

- compares %u:%g:%a with 0:0:700 and 0:0:600;
- reports the observed numeric tuple on rejection without exposing verifier contents;
- uses chown 0:0 and install -o 0 -g 0 for every ownership operation that can execute before users activation;
- keeps the post-users consumer check numeric for consistency and future ordering safety.

This is not merely a compatibility relaxation. A hostile NSS fixture mapping the names root:root to nonzero IDs made the old predicate accept a non-root inode; the numeric predicate rejected it. Numeric validation enforces the kernel ownership invariant more strictly.

Nixpkgs briefly used the same numeric-check/name-diagnostic distinction for installer-root validation, although that general check was reverted because non-root-owned image/chroot roots can be legitimate. This secret path has a repo-specific root-ownership invariant, so the revert does not contradict this repair. [Source 6: added numeric precedent](#), [Source 7: scope-related revert](#)

Verification

- Formal RED before production edit: the new empty-target-NSS lifecycle case failed with the exact old error. [RED evidence](#)
- GREEN after repair: existing verifier and missing-directory/sentinel migration both pass with explicit files-only NSS and empty passwd/group. [GREEN evidence](#)
- Correct-mode wrong UID and wrong GID are independently rejected for both directory and file. [Numeric owner matrix](#)
- Mutation suite directly snapshots tracked working-tree edits through an isolated index and kills both dependency-order, %u-%U, %g-%G, named chown/install, lifecycle, helper, symlink, hash-shape and secret-scan mutants without altering the real index. [Mutation log](#)
- Helper, lifecycle, secret scan, config evaluation, Bash syntax, ShellCheck, documentation shell syntax and Git whitespace checks pass.
- `nix flake check --all-systems --no-build` passes.
- Full x86_64 NixOS system builds at `/nix/store/llvz0c9whmx6d90ig81ki05354i8np4w-nixos-system-nixos-26.11.20260705.d407951`; its generated activation script contains the numeric operations in the expected order. [Full matrix](#)

Safe recovery of the failed target

Do **not** rerun the password helper unchanged merely to recover this failure: its normal `nixos-anywhere` invocation includes destructive Disko formatting.

The guide now includes a secret-safe diagnostic block that compares numeric and named metadata from the ISO and direct target chroot. If `/mnt` remains correctly mounted and the staged verifier is valid, rerun only `nixos-anywhere`'s install phase from the corrected temporary checkout. If it is no longer mounted, pinned `nixos-anywhere` documents `--disko-mode mount` as the non-formatting recovery mode. [Source 8: pinned Disko recovery guide](#)

NixOS also explicitly supports correcting a failed configuration and rerunning `nixos-install` on mounted target filesystems. [Source 9: pinned NixOS recovery guidance](#)

The retained installer checkout `/tmp/tmp.mmc4BwexfA/nix-config` has been synchronized with the corrected module/helper/docs/tests while preserving its machine-specific Micron disk path.

Remaining limitations

- The agent had no authorized connection to the live target, so the failed target's actual `/mnt` tuple and a hardware end-to-end retry remain unobserved.
- A full installer VM run was designed but not executed; exact generated validator/system-boundary tests, full system build, and independent container/user-namespace matrices passed.
- `--phases install` is safe only while `/mnt` is correctly mounted. Otherwise use the documented mount-mode recovery after re-verifying the disk path.
- No verifier or plaintext password was printed or stored in research artifacts.

- Five independent final-review lanes (goal, QA, code, security, and context) all passed after their findings were fixed. [Review report](#)